2022 19th International Bhurban Conference on Applied Sciences and Technology (IBCAST) | 978-1-6654-6051-4/22/$31.00 ©2022 IEEE | DOI: 10.1109/IBCAST54850.2022.9990042

A Requirement Based Approach to Test Case Prioritization for Regression Testing

Misbah Naheed
Department of computer science
Capital University of Science and Technology
Islamabad, Pakistan.
misbah.naheed47850@gmail.com

Aamer Nadeem
Department of Computer Science
Capital University of Science and Technology
Islamabad, Pakistan
aamer.nadeem@cust.edu.pk

Qamar uz Zaman
Department of Computer Science
Capital University of Science and Technology
Islamabad, Pakistan
qamar.zaman@cust.edu.pk

***Abstract*— Testing is the strategy wherein a program is executed fully intent on tracking down bugs. Whenever modification is done it is verified again to ensure that there are no new flaws have emerged in the previously tested code and that it continues to function effectively. This kind of testing is called regression testing. Original program may have many test cases and executing the whole test suite may increase the cost of testing. Regression test selection, test suite minimization and the test case prioritization are three strategies of regression testing which lower cost of regression testing. In test case selection and minimization techniques, only few test cases are chosen and the leftover test cases are squandered. The risk is that useful test cases with respect to fault detection may be discarded while in prioritization the software tester prioritizes the test and test cases that are more suitable to detect errors early are run first. There is no risk of important test case elimination in this technique as compared to other techniques. Therefore, test case prioritization is more effective and time saving. In test case prioritization technique important test cases are placed first in the prioritization list. Maximize early fault detection is goal of prioritization. In prioritization, technique test cases are ranked according to some coverage criteria. Existing techniques use total coverage strategy for prioritization of test cases by considering the total number of requirements covered. There may be more than one test case that covers the same requirements. Therefore, there is a possibility to repeat the test cases that cover same requirements, which is not desirable. In this research, we offer a new test case prioritizing technique based on prioritized requirements. This algorithm uses the "Additional" coverage strategy. Test case that covers the most requirements and has the highest weight is given high priority by our algorithm. Our suggested prioritization algorithm outperforms than existing techniques as it gives earlier coverage of requirements.**

Keywords— ***Test case prioritization, Prioritized Requirements, Software Testing***

I. INTRODUCTION

Software testing is an important practice for improving software quality. The goal of testing is to uncover defects from software and remove them [1]. Although testing removes bugs from the system but it does not ensure that the system is completely free of bugs. In software quality assurance, testing is an integral part when software is updated and modified so new modified and updated test suite may be required. Different types of testing are used to remove different types of errors; errors could be design errors, specification errors and input errors [2]. White box testing, Black box testing [3], and Grey box testing [4] are three types of software testing. Regression testing is a kind of software testing that is used to guarantee that the previously tested code is still functional and free of new bugs. It instils confidence in the developer that new changes do not disturb the unchanged part of software. Many changes may occur at maintaince phase where the system is corrected. There are two types of maintaince: corrective maintaince and progressive maintaince. Based on these types of maintaince two types of regression testing can be done. In progressive regression testing the specifications are updated by removing a module then testing is performed to test this updation in software. While in corrective regression the errors are found and fixed as the specifications remain unchanged [5].

Test suites increase in size as software is upgraded. Running entire test suite is expensive for updated system. More time is required to test with the whole test suite [6]. To cut the cost of regression testing three techniques are employed.

- **Test suite Minimization:**
  This technique shrinks test suite by eliminating duplicate test cases from test suite. Various coverage criteria are employed for minimization and test cases that fulfil the coverage criteria are selected and remaining test cases are discarded [7].
- **Regression test Selection:**
  The modified module of the system find out then choose the test cases, which cover the modified module and discard leftover test cases. In selection we have to analyze the dependencies between the modules and only updated modules test cases are chosen; remaining test cases are discarded the chances of important test case elimination is more [8].
- **Test case Prioritization**
  It finds the optimal sequence of test cases so that goal of testing is obtained. The test case prioritization technique puts test cases in a prioritization list so that the most essential test cases appear first.
  Prioritization's goal is to maximize early fault detection [9].

Different coverage criteria are used for prioritization. The entities of source code, such as branches, statements, and functions, can be used as criterion. [10]. Priority is given to the test case that covers the most entities. [11]. The coverage criteria can be specification based, customer priority or

978-1-6654-6051-4/22/$31.00 ©2022 IEEE

developer priority etc [12]. There are two coverage strategies "Total" and "Additional" strategy [13].

"Total" strategy prioritizes the test cases according to maximum number of entities covered. "Additional" technique selects test case with the highest degree of coverage and covers entities not previously covered by any test case. It's possible that two or more test cases in a total strategy will have the same coverage entities which may result duplication of entities being tested [13].Whereas additional coverage strategy reduces the duplication by ensuring the coverage of each entity by each of the test case at least once.

Prioritization can be done using black box testing techniques depending on the amount of requirements covered or the total priority of each test case. Some test case prioritization techniques based upon requirement considers only the number of requirements covered by the test case and do not consider the requirement's priority total weight. Because priority is not taken into account, the requirement with the highest importance may be placed towards the end of the test suite. The amount of requirements addressed by each test case is the sole factor taken into account. Some techniques prioritize test cases based on requirement's total weight. They only consider the requirement's total weight priority and do not consider the number of requirements that each test case covers. There is a possibility that we have a requirement whose weight is less and due to total weight formula it will be at the end of test suite then its execution will be late or at the end. Existing techniques do not consider both criteria using additional strategy.

We propose a prioritized requirements based prioritization technique that takes into account both priority weight and number of requirements covered by test case. The remainder of the paper is structured as follows: Different requirements-based prioritization approaches are explained in Section 2. The proposed algorithm is described in Section 3. Section 4 gives an example to demonstrate how the suggested algorithm works, and Section 5 draws the conclusion of paper.

II. RELATED WORK

To increase the testing efficiency test case prioritizing approaches arrange test cases in the test suite so that the most critical ones are run first. Finding the faults early saves the cost and helps to deliver the software on time [9].

Total and additional are two basic prioritization strategies [13]. The total approach takes into account the total number of entities covered by each test case, whereas the additional strategy takes into account the test case which covers the entities not covered so far by any test case and also selects test cases with high coverage degree [9].

Requirements based test case prioritization is one of the black box prioritization technique in which system specifications are used to define the prioritization criteria. S.Hema and W.Laurie [15] proposed prioritization strategy based upon three factors: customer assigned priority, requirements Complexity and requirements volatility. The value of Customer assigned priority (CP) is assigned by the customer between the range of 1 to 10 and the requirement complexity (RC) is decided by the developer by assigning a value of (1 to 10). These factor values are used to calculate the weighted prioritization (WP) factor which helps to order the test cases for requirements in such a way that high WP value test case is executed first than others. This prioritization technique minimize the effort for test case prioritization as compare to other techniques i.e. coverage based technique. This technique gives high value to customer and improves the software quality as severe faults are identified earlier [15].

R.Kavitha et al. [16] proposed an algorithm selects the factor according to prioritization goal. It assigns the factor value for all requirements by concerned people then calculates the weight. In total weight of every test case is arranged in descending sequence. The higher weight test case will run first prior to others.

R Krishhnamoorthi and M Sahaaya Arul [17] present six factors for requirements prioritization for testing. Two analyses verify their prioritization technique. The "number of flaws detected" is the subject of the first analysis and "number of test cases executed to detect the faults" is the subject of second analysis. These factors prioritize the system test cases made up of customer priority, changes in requirements, implementation complexity, completeness, traceability and fault impact. For customer satisfaction the focus on customer's requirement is high. High priority will be given to the requirements with highest value; a value of 10 indicates the highest priority and 1 scale shows lowest priority. Similarly, a value of ranging from 0 to 10 shows the implementation complexity. The number of times a requirement is changed is showed on a scale of 1 to 10. It is estimated that 50% of errors occur during requirements phase. Fault impact helps to determine those requirements which are reported as failure by the customer. Completeness (CT) checks that whether each requirement is completed; the condition of function is fulfilled or not. If a requirement is considered for reuse then a value from 0 to 10 is given by the customer. Traceability (TR) enhances the quality of software. When the requirements are reused then the tester gives a value from 0 to 10. Calculate the weight of each test case by adding the weight of the requirement. Test cases are arranged by weight in descending sequence. The higher-weighted test cases are executed first. [17].

S.Raju and G.V.Uma [18] proposed a new algorithm in which the eight factors are selected by keeping in view the prioritization goal. By using the traceability approach each requirement is mapped on a test case. Each test case's weight is determined, and then the test cases are ranked in ascending order.

M.J.Arafeen and D.Hyunsook [19] state that a system may have many requirements which are of high priority. Not all requirements are equally significant, and some have a higher priority than others. For severe fault detection important test cases executed first. The main objective of requirements clustering is to group test cases in such an order to improve test case prioritization. Utilizing requirement information increases effectiveness of test case prioritization. In requirements clustering the text mining approach is used in which useful words are extracted from text. By using these words, relevant

requirements are clustered. Within the cluster using code complexity test cases are prioritized. In the next step, code modification information and customer assigned requirement priority are used to prioritize the cluster. At the end different selection methods are used to select the test cases from each cluster.

H.Kumar et al. [20] proposed hierarchical test case prioritization (HSTCP) in which the requirements are prioritized on the basis of twelve factors by assigning the weight to each requirement. The factors are customer assigned, developer assigned, requirement volatility, fault proneness, expected fault, implementation complexity, execution frequency, traceability, show stopper requirement, penalty, cost and time. These twelve factors are scaled on 1 to 10 values. Prioritized list of requirements is obtained on the first level. At the second level, the mapping is drawn between requirements and corresponding modules. If there is more than one corresponding module then modules are prioritized based on Cyclomatic complexity and non dc path. Module with higher priority value is given highest priority than other modules. At the highest level, test cases are sorted according to four criteria: test impact, test case complexity, requirement coverage, and dependency. The resulting test suite is the prioritized test suite.

M.Thillaikarasi and Dr.Seetharaman [21] suggest an algorithm for prioritizing requirements build upon six factors. The factors are customer allotted priority, developer observed complexity, changes in requirements, fault impact, completeness and traceability. Values are assigned on a scale of 1 to 20 to these factors. The factor value 20 indicates the highest priority of the requirement. Then weighted prioritization value (WPV) is calculated by using priority factor value and priority factor weight. By using weighted prioritization value (WPV) in the test suite, each test case is prioritized. Total coverage strategy for prioritization of test case based on prioritized requirements only considers total number of requirements covered. There may be more than one test case that covers the same requirements. So there is repetition of the requirements covered which is not desirable. Additional coverage strategy removes this drawback by assigning the priority based upon the coverage of uncovered requirements. If a test case covers only one requirement then that test case will have less total weight as compared to other test case that covers more than one requirement. Due to total weight formula the test case with less weight shifted to end of priority list although it cover important requirement. Some test case prioritization techniques based upon requirement's priority; consider only the number of requirements covered by the test case and did not consider the requirement's priority total weight. The requirement whose priority is higher may goes at the end of test suite because the requirement's priority is not considered only number of requirements covered by each test case is considered.

Some techniques are prioritizing the test cases by considering the requirement's total weight. They only consider the requirement's total weight priority and it did not take into account how many requirements each test case covered. There is a possibility that we have a requirement whose weight is less and due to total weight formula it will be at the end of test suite then its execution will be late or at the end.

III. PROPOSED APPROACH

Prioritized requirements based test case prioritization algorithm will use the mapped data of test cases and prioritized requirements as input and create a priority list of test cases as output. Requirements are prioritized on two factors customer priority (CP) and developer priority (DP). Developer assigns customer assigns customer priority value and developer priority. Total requirement priority value calculated by using formula RP=CP+DP.Algorithm takes inputs of test cases and prioritized requirements along with their priority value. Mapping of test case with requirements is done then total weight of test cases is calculated.

Procedure for prioritizing regression test cases

Input:

1. ***T`:*** *Set of Test Cases for P`*
2. ***Reqcov:*** *set of prioritized requirements in P covered by the tests in T`*
3. ***Cov(t) :*** *set of requirements covered by executing P against t.*
4. ***X`:*** *a temporary set of regression tests used for calculation.*
5. ***Req count:*** *count the number of requirements covered by test case t.*
6. ***Weight(t):*** weight of test case t.

Output:

PrT: *A sequence of test cases.*

Procedure:

Step 1: X`=T`. Find t in X` such that
$\forall t` \in X`$ cov(t).weight $\geq$ cov(t`).weight

Step 2: while X` $\neq \emptyset$ and req count $\neq 0$

Step 2.1: *Append t to prT*
X`= X` \ (t), Reqcov=Reqcov\ cov (t)

Step 2.2: *calculate X`.weight(t) such that*
Weight(t)= Weight(t)\ ReqCov.

Step 2.3: *calculate Reqcount(t) such that*
Req Count= Req Count \ ReqCov.

Step 3: Append to PrT any remaining test in X`.

Fig.1. Proposed Algorithm

IV. IMPLEMENTATION ANALYSIS AND DISCUSSION

The algorithm implemented in C sharp language using visual studio tool (2010) and database is maintained in Microsoft SQL Server Management Studio (2008). The tool firstly takes the prioritized requirements along their weights and test cases. The mapping of test cases to the number of requirements covered by each test case is accomplished. By using greedy prioritization algorithm. Greedy algorithm mostly concern with test case prioritization problem that concentrate on selecting the best test case during test case prioritization.

Muthusamy algorithm for test case prioritization is considered for comparison.

First case study is about the project "Process Flow Manager" consists of 35 prioritized requirements and 19 test cases. The mapping of prioritized requirements with test cases is given in table I:

TABLE I. Test case mapping with prioritized requirements

Process Flow Manager		
Test case	**Covered requirements**	**Total weight of test case**
t_1	r_1, r_2	15+13=28
t_2	$r_1, r_3 r_4$	15+14+11=40
t_3	r_3, r_5, r_6	14+10+13=37
t_4	r_3, r_6, r_7	14+13+11=38
t_5	$r_1, r_7 r_8, r_9$	15+11+13+11=50
t_6	$r_9, r_{10}, r_{11}, r_{12}$	11+10+10+12=43
t_7	$r_{11}, r_{13}, r_{14}, r_{15}$	10+11+13+10=44
t_8	r_{14}, r_{15}, r_{18}	13+10+10=33
t_9	$r_1, r_{16}, r_{17}, r_{18}$	15+12+12+10=49
t_{10}	r_{17}, r_{19}	12+11=23
t_{11}	r_{19}, r_{20}, r_{21}	11+11+12=34
t_{12}	r_{21}, r_{22}, r_{23}	12+12+12=36
t_{13}	$r_8, r_{23} r_{24}, r_{33}$	13+12+12+13=50
t_{14}	r_8, r_{25}, r_{26}	13+13+13=39
t_{15}	r_9, r_{27}	11+14=25
t_{16}	$r_9, r_{28} r_{29}$	11+11+16=38
t_{17}	$r_9, r_{30} r_{31}$	11+11+13=35
t_{18}	r_9, r_{32}, r_{33}	11+13+13=37
t_{19}	$r_1, r_9, r_{33} r_{34} r_{35}$	15+11+13+12+12=63

Comparison is done on the basis of "number of requirements covered" and "total weight of test case". The below graph shows that in existing test case prioritization technique to cover all requirements we have to execute whole test suite. The prioritized list of existing test case prioritization is Prt{t_{19},t_5,t_{13},t_9,t_7,t_6,t_2,t_{14},t_{16},t_4,t_3,t_{18},t_{12},t_{17},t_{11},t_8 ,t_1,t_{15}, t_{10}} Because in existing techniques they only consider total weight of test case and do not consider priority of requirement. While in our test case prioritization technique the requirement's priority is also considered. New test case prioritization list is Prt{t_{19},t_7,t_{14}, t_{12},t_9, t_{16},t_2, t_{17},t_4,t_6,t_{11},t_{15}, t_{18} ,t_1 , t_{13},t_3, t_5 ,t_{10} , t_8}.

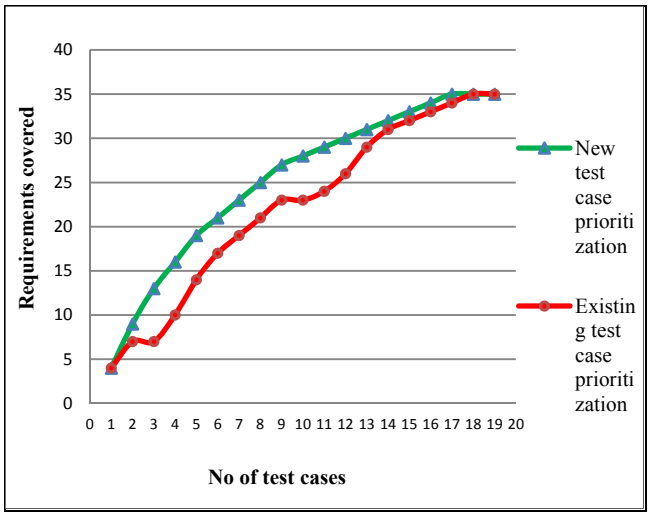

Fig.2. Comparison of number of requirement covered for case study 1

The fig 2 shows that in existing prioritization technique to cover all the requirements we have to execute 18 test cases whereas in new test case prioritization technique only 16 test cases are executed to cover all requirements. The green line which shows new test case prioritization gives highest coverage as this technique cover requirements early.

Our proposed technique gives maximum coverage of requirements earlier as compared to existing technique. The test case that covers maximum number of requirements is selected first. There is no repetition of requirements in our technique.

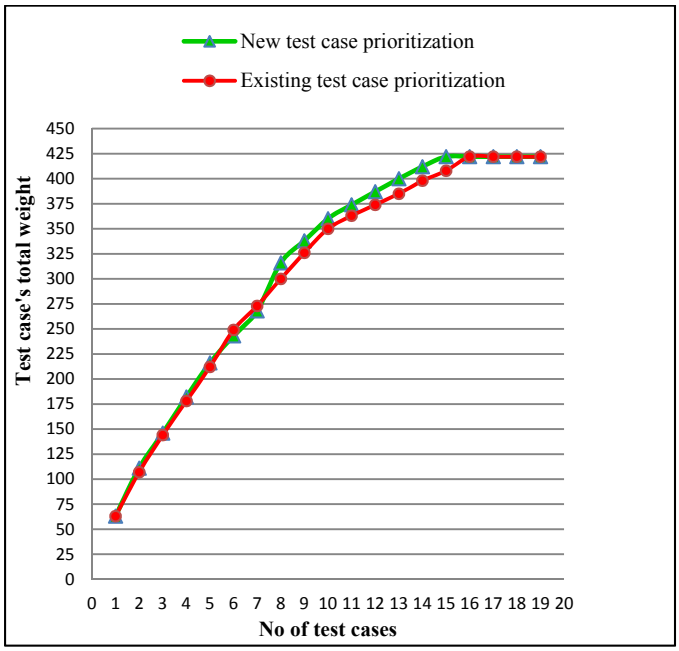

Fig.3.Comparison of total weight priority for case study 1

Fig.3 depicts that the new test case prioritization technique gives more weight coverage as at t_3 total weight coverage is 146 as compare to existing technique t_3=144. After test case t_3 the total weight coverage is greater by new technique as compared to existing technique. At test case t_{17}, t_{18}, t_{19} the lines of graph meet each other because the total weight coverage is achieved by earlier sixteen test cases.

Second case study is about "Automatic teller machine" consists of 20 prioritized requirements and 16 test cases. ATM allows a person to check account balances, withdraw or deposit money, print a statement of account transactions.The mapping of prioritized requirements with test cases is given below:

Table II. Case study II test case mapping with prioritized requirements

Automatic teller machine		
Test case	**Covered requirements**	**Total weight of test case**
t_1	r_2, r_3	13+11=24
t_2	r_1, r_4	12+11=23
t_3	r_2, r_3, r_4, r_5	11+13+11+16=51
t_4	r_6, r_7, r_8	13+14+10=37
t_5	r_3, r_8	11+10=21

t_6	r_{10},r_{11}, r_{12}	17+16+14=47
t_7	r_7,r_9, r_{10}, r_{11},r_{12}	14+11+17+16+14=72
t_8	r_{10},r_{12}	17+14=31
t_9	r_3,r_{13}	11+14=25
t_{10}	r_{14},r_{15}	8+13=21
t_{11}	r_{15},r_{16}	13+9=22
t_{12}	r_{14}, r_{17}, r_{18}	8+15+13=36
t_{13}	r_{17} ,r_{18}	15+13=28
t_{14}	r_{18}, r_{19}, r_{20}	13+12+10=35
t_{15}	r_{17}, r_{18}, r_{19}	15+13+12=40
t_{16}	r_{19}, r_{20}	12+10=22

Comparison is done on the basis of "number of requirements covered" and "total weight of test case". The fig 4 shows that in existing test case prioritization technique to cover all requirements we have to execute whole test suite. The prioritized list of existing test case prioritization is Prt{t_7,t_3,t_6, t_{15},t_4,t_{12}, t_{14},t_8,t_{13}, t_9,t_1,t_2,t_{11},t_{16},t_5, t_{10}}. The repetition of requirements exist in it as test case t_7 covers r_7,r_9 ,r_{10}, r_{11},r_{12} and t_6 covers same requirementsr_{10}, r_{11},r_{12} ; test case t_8 also covers r_{10}, r_{12}.Because in existing techniques they only consider total weight of test case and do not consider priority of requirement. While in our test case prioritization technique the requirement's priority is also considered. New test case prioritization list is Prt{t_7,t_3,t_{15}, t_4,t_{11}, t_9,t_2, t_{14},t_{12},t_{13},t_{10},t_{16}, t_1 ,t_8,t_5 ,t_6}.

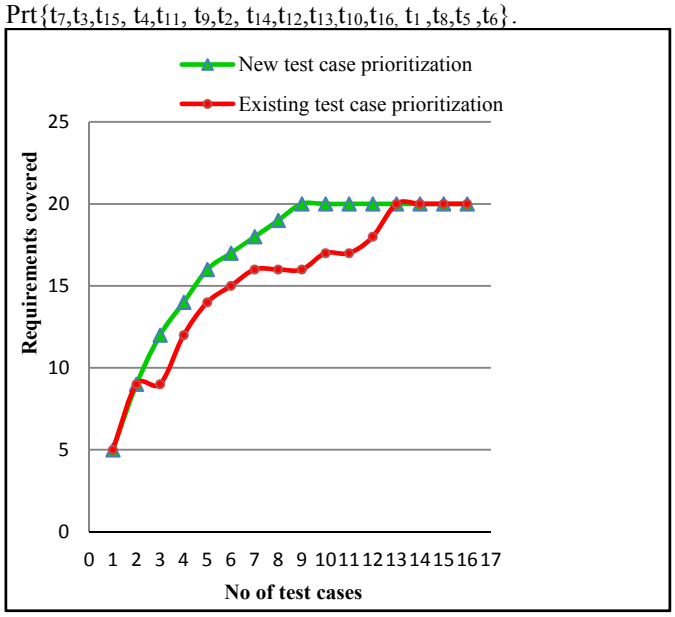

Fig.4. Comparison of number of requirement covered for case study 2

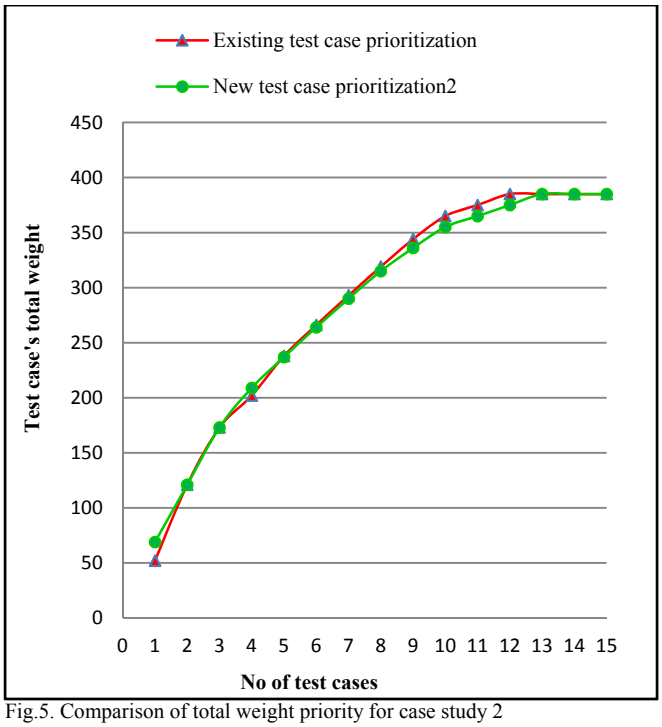

Fig.5. Comparison of total weight priority for case study 2

At t_6, t_7, t_8 the weight coverage is higher by new test case prioritization. The difference is very less between existing and new technique because all requirements are covered earlier only by nine test cases; remaining seven test cases will not be executed. If the requirements are covered earlier by less number of test cases then difference would be very less.

V. CONCLUSION AND FUTURE WORK

Software is retested whenever the modification is done to check that previously tested code does not create any new defects. To reduce the cost of regression testing there are three techniques used; test case minimization, test suite selection and test case prioritization. Test case prioritizing strategy is superior to other techniques because no test case is excluded. In prioritization test cases are prioritized using various coverage criteria. Two coverage strategies are use "total" and "additional". Additional coverage is preferable to total coverage because it includes not only the number of entities covered by each test case but also the additional entities that remain uncovered by previously selected test cases. It reduces the of entities and ensures coverage of each entity. The total technique is used in existing prioritizing strategies to prioritise test cases, and "total" refers to the total number of entities considered by each test case. Existing prioritization criterion is based on total weight priority or number of requirements covered by each test case. Our proposed test case prioritization based on prioritized requirements uses both criteria, the number of requirements covered by each test case is used as primary criterion and total weight priority is used as secondary criterion using additional strategy.

We intend to extend our algorithm in the future by using different criteria as existing test case prioritization prioritize the test cases on the basis of the number of requirements

covered or considering the priority of requirements. However, they do not consider, that a requirement may not be fully addressed by a single test case. Complex requirements are covered by more than one test case. Therefore, existing requirements based test case prioritization approaches may give lower priority even to those test cases which cover high-priority or critical requirements

REFERENCES

[1] R. Kaur Chauhan and I. Singh, "Latest research and development on software testing techniques and tools", International Journal of Current Engineering and Technology, vol. 4, no. 4, ISSN 2347 - 5161, 2014.

[2] D. Jeffrey and N. Gupta, "Experiments with test case prioritization using relevant slices", Journal of Systems and Software, vol. 81, no. 2, pp. 196-221, 2008.

[3] I. Hooda and R. Singh Chhillar, "Software test process, testing types and techniques", International Journal of Computer Applications, vol. 111, no. 13, pp. 10-14, 2015.

[4] R. Saxena and M. Singh, "Gray box testing: proactive methodology for the future design of test cases to reduce overall system cost", Journal of Basic and Applied Engineering Research, vol. 1, 8, no. 2350-0255, 2014.

[5] K.Leung and L.White. "Insight into regression testing", proceedings of International Conference on Software Maintenance (ICSM), IEEE Computer Society Press, pp, 60–69 1989.

[6] S. Yoo and M. Harman, "Regression testing minimization, selection and prioritization: A survey", in Software Testing, Verification, Rel., vol. 22, no. 2, pp. 67–120, Mar. 2012.

[7] G.M.R, "Computers and intractability: a guide to the theory of np-completeness", W.H.Freeman and company: New York, 44(2), pp.340, 1979.

[8] G.Rothermel and M.Harrold "A framework for evaluating regression test selection techniques" , Proceedings of 16th International Conference on Software Engineering, 1994.

[9] S. Elbaum, A. Malishevsky and G. Rothermel, "Test case prioritization: a family of empirical studies", IEEE Transactions on Software Engineering, vol. 28, no. 2, pp. 159-182, 2002.

[10] G. Rothermel, R. Untch, Chengyun Chu and M. Harrold, "Test case prioritization: an empirical study", Proceedings IEEE International Conference on Software Maintenance - 1999 (ICSM'99). 'Software Maintenance for Business Change' (Cat. No.99CB36360), 1999.

[11] Y. Fazlalizadeh, A. Khalilian, M. Azgomi and S. Parsa, "Prioritizing test cases for resource constraint environments using historical test case performance data", 2009 2nd IEEE International Conference on Computer Science and Information Technology, 2009.

[12] C. Henard, M. Papadakis, M. Harman, Y. Jia and Y. Le Traon, "Comparing white-box and black-box test prioritization", Proceedings of the 38th International Conference on Software Engineering - ICSE '16, 2016.

[13] G. Rothermel, R. Untch, Chengyun Chu and M. Harrold, "Prioritizing test cases for regression testing", IEEE Transactions on Software Engineering, vol. 27, no. 10, pp. 929-948, 2001.

[14] C.Henard, M.Papadakis, M.Harman, M.Jia and Y.L.Traon (2016)."Comparing white-box and black-box test prioritization",in Proceedings of the 38th International Conference on Software Engineering (ICSE), ACM, 2016.

[15] S.Hema and W.Laurie, "Requirements-based test case prioritiation" Advanced Computing and Communication (CACC),2008.

[16] R.Kavitha, V.R.Kavita and Dr.N.Suresh Kumar, "Requirement based test case prioritization" IEEE Computer Society,2010.

[17] R Krishhnamoorthi and M Sahaaya Arul , "Factor oriented requirement coverage based system test case prioritization of new and regression test cases." Information and Software Technology Journal**,** PP.799-808, 2009.

[18] S.Raju and G.V.Uma, "Factors oriented test case prioritization technique in regression testing using genetic algorithm" European journal of Scientific Research pp. 389-402, 2012.

[19] M.J.Arafeen and D.Hyunsook, "Test case prioritization using requirements based clustering". International Conference on Software Testing. IEEE Computer Society Press, pp, 312–321, 2013.

[20] H.Kumar, V.Pal and N.Chauhan, "A hierarchical system test case prioritization technique based on requirements." 13th Annual International Software Testing Conference, pp, 4–5, 2013.

[21] M.Thillaikarasi and Dr.Seetharaman, "Measuring the effectiveness of test case prioritization techniques based on weight factors" department of computer science, IEEE Computer Society pp. 41–51, 2016.